

YipCommerce Case Study

A Laravel/PHP e-commerce application demonstrating storefront, checkout, admin operations, and Smarty template integration.

What Was Built

YipCommerce Case Study is a basic but polished e-commerce web application with a storefront, shopping cart, checkout, customer order history, and admin management area. It is intentionally scoped as a case-study project rather than a large marketplace.

The application includes:

- Product listing, search, and detail pages
- Session cart with stock validation
- Authenticated checkout
- Order and order item persistence
- Customer order history
- Admin product management
- Admin order management
- Responsive Tailwind UI
- Smarty-rendered about page

Why Laravel/PHP

Laravel was chosen because it is a mature PHP MVC framework with strong conventions around routing, controllers, middleware, validation, Eloquent ORM, migrations, seeders, and testing. YipOnline's custom YIP Framework is described as Laravel-inspired and modular, so Laravel is a practical fit for demonstrating transferable PHP full-stack skills.

MVC Structure

The application follows Laravel's MVC structure:

- Models: `User`, `Product`, `Order`, and `OrderItem`
- Controllers: public storefront controllers, auth controllers, and admin controllers
- Views: Blade templates for storefront/admin screens and one Smarty `.tpl` template
- Services: `CartService`, `OrderService`, and `SmartyRenderer`
- Middleware: `AdminMiddleware` for role-based access control
- Form Requests: checkout, product, order status, and payment status validation

This keeps routing, request validation, business rules, persistence, and presentation in separate layers.

Database Structure

The database contains:

- `users`: customer/admin accounts with a `role` field
- `products`: catalog records with slug, description, price, image, stock, and status
- `orders`: customer order header, delivery details, payment method, status, payment status, and totals
- `order_items`: individual purchased products, quantity, unit price, and line total

Relationships:

- User has many orders
- Order belongs to user
- Order has many order items
- Order item belongs to order
- Order item belongs to product
- Product has many order items

Cart And Checkout Flow

The cart is session-based. `CartService` stores product IDs and quantities in the session, then hydrates current product data from the database whenever cart details are displayed or used.

Checkout requires authentication. `OrderService` validates the cart, creates an order, creates order items, reduces product stock, and clears the cart inside a transaction.

Checkout prevents:

- Empty cart checkout
- Adding inactive products
- Adding or checking out quantities above available stock
- Negative quantity manipulation

Admin Order Management Flow

Admin routes live under `/admin` and require both `auth` and `admin` middleware. Admin users can:

- View dashboard metrics
- See recent orders
- List all orders
- Open single order details
- Update order status
- Update payment status

Supported order statuses are `pending`, `processing`, `completed`, and `cancelled`. Payment statuses are `unpaid` and `paid`.

Security Measures

- Password hashing for all users
- CSRF protection on all forms
- Auth middleware for checkout and order history
- Admin middleware for admin routes
- Form Request validation
- Customer order authorization checks
- Eloquent/query builder instead of raw SQL
- Blade output escaping
- Smarty escape modifiers for dynamic template variables
- No sensitive secrets exposed in frontend code

Smarty Integration

The `/about` route demonstrates Smarty integration. `HomeController` sends a simple view model into `SmartyRenderer`, which configures Smarty template, compile, and cache directories. The rendered Smarty fragment is then placed inside the normal Blade layout.

This mirrors how a Laravel-inspired custom framework could support multiple template engines while keeping rendering behind a service boundary.

Relation To YipOnline

YipOnline's platform work requires scalable, maintainable business applications. This project maps to that environment by showing:

- Modular Laravel-inspired structure
- Clear separation of MVC concerns
- Admin workflows for business operations
- Data integrity during checkout
- Role-based access control
- A path for Smarty compatibility
- A codebase that can grow without starting from a tangled prototype

Future Improvements

- Payment gateway integration
- Order confirmation emails
- Product categories and filtering
- Database-backed carts
- Product image cleanup on replacement
- Policies for more granular authorization
- Inventory audit logs
- Browser-level end-to-end tests